
1. Product-by-Product Support Breakdowns

A. PostgreSQL Database

- **Ticket Examples & Pain Points:** Support tickets in this category often feature error codes like 57M01 (too many clients) or complaints that database queries are timing out during traffic spikes. Developers encounter these issues when client applications directly open too many database connections without pooled management, or when running SELECT queries on large, unindexed tables, leading to high CPU usage and disk I/O bottlenecks.
- **Solution:** Migrate application connection strings to use [Supabase's built-in connection pooler \(Supavisor\)](#) in **Transaction mode** for high-concurrency client requests. Additionally, apply B-tree or GIN indexes to frequently queried columns, analyze query performance using EXPLAIN ANALYZE, and configure statement timeouts to auto-terminate runaway queries before they exhaust database resources.

B. Authentication

- **Ticket Examples & Pain Points:** Common tickets include redirect_uri_mismatch errors during OAuth logins (e.g., Google or GitHub), mobile deep-link callbacks failing silently, or transactional magic-link emails hitting default rate limits and landing in spam folders. Pain points stem from missing wildcard redirect URLs across staging and production environments, mishandled PKCE code exchanges on single-page apps (SPAs) and mobile devices, or relying on Supabase's shared default SMTP server for production app traffic.
- **Solution:** Explicitly configure all deployment and deep-link URLs under **Auth Settings > Additional Redirect URLs**, ensure frontend SDKs are set up with PKCE flow enabled for mobile/SPA architectures, and connect a custom SMTP provider (such as Resend or SendGrid) within Supabase Auth to handle email delivery and bypass default rate limits.

C. Storage

- **Ticket Examples & Pain Points:** Developers frequently open tickets regarding 403 Access Denied errors on file uploads, 413 Payload Too Large responses, or client-side CORS errors when retrieving uploaded assets. These issues usually stem from uploading files without explicit Row-Level Security (RLS) policies on the storage.objects table, exceeding default bucket file size limits, or attempting to fetch private bucket assets using public URLs instead of authenticated session tokens.
- **Solution:** Define explicit RLS rules on the storage.objects table allowing authenticated users (auth.role() = 'authenticated') to execute INSERT and SELECT actions on target buckets. Configure valid CORS rules in bucket settings to allow client domain origins, set file size and MIME-type constraints directly on the bucket configuration, and utilize temporary **Signed URLs** for private file downloads.

D. Realtime

- **Ticket Examples & Pain Points:** Realtime support tickets often report clients not receiving INSERT/UPDATE broadcast payloads, missing event notifications, or WebSockets unexpectedly dropping connections under heavy traffic. Pain points occur because target database tables were never explicitly added to the supabase_realtime PostgreSQL publication, or because Row-Level Security (RLS) rules silently filter out event notifications for unauthorized subscribers.
- **Solution:** Add missing tables to the publication via SQL (ALTER PUBLICATION supabase_realtime ADD TABLE table_name;), verify that RLS policies allow target user roles to read the broadcasted rows, and implement client-side exponential backoff and heartbeat listeners to handle socket reconnections cleanly.

E. Edge Functions

- **Ticket Examples & Pain Points:** Typical tickets involve 504 Gateway Timeout errors during long-running tasks, runtime crashes due to missing Deno.env secrets, or browser CORS preflight (OPTIONS) failures when hitting function endpoints. Developers face these hurdles because Edge Functions operate on isolated serverless runtimes (Deno) with strict execution time limits (e.g., 10–15 seconds wall-clock time) and require explicit CORS headers in response payloads.
- **Solution:** Add standard CORS header handling and preflight OPTIONS route responses at the top of the TypeScript entry point, set environment secrets via the Supabase CLI (supabase secrets set KEY=VALUE), and offload long-running background jobs to queue systems or webhooks rather than holding open single HTTP request/response loops.

F. Vector Database

- **Ticket Examples & Pain Points:** Tickets in this domain center around memory-exhaustion crashes during vector index creation or dimension mismatch exceptions (e.g., inserting 1536-dimension OpenAI vectors into a 768-dimension column). Pain points arise when executing unindexed vector searches across thousands of documents, which triggers slow exact-nearest-neighbor scans that consume high RAM and lock database worker threads.
- **Solution:** Match vector column dimensions precisely to the embedding model's outputs, build **HNSW** (Hierarchical Navigable Small World) or **IVFFlat** indexes using appropriate distance operators (vector_cosine_ops or vector_l2_ops), and tune index query parameters (ef_search / lists) to balance vector search accuracy, memory footprint, and query latency.

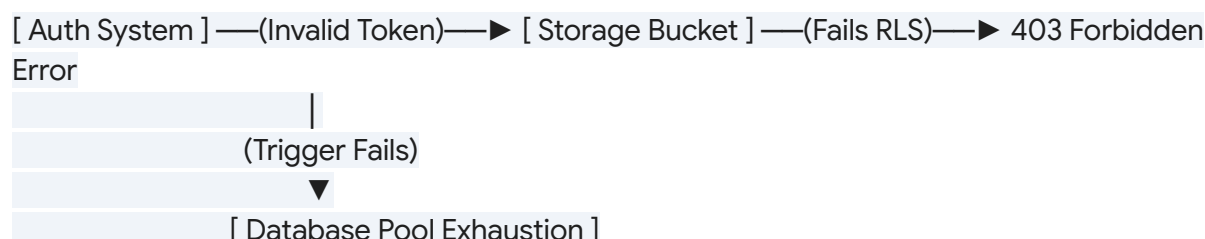
G. Row-Level Security (RLS)

- **Ticket Examples & Pain Points:** RLS generates the highest volume of support tickets, predominantly featuring queries returning empty [] arrays with zero errors thrown, or database panics reporting infinite recursion detected. Pain points occur because enabling RLS without explicit policies defaults to denying all access, while policies that cyclically query related foreign tables trigger infinite loops during execution.

- **Solution:** Construct targeted SQL policies utilizing `auth.uid()` and `auth.jwt()` helper functions, convert cyclic subquery checks into `SECURITY DEFINER` database functions to safely evaluate policies without triggering infinite loops, and add indexes to foreign key columns evaluated inside RLS policies to maintain query performance.

2. Connected Multi-System Pain Points & Bug Scenarios

Real-world production bugs often span multiple Supabase services simultaneously. Here are 4 complex interconnected issue patterns:



1. Profile Avatar Upload Fails Silently (Auth + Database Triggers + Storage RLS)

- **Scenario:** A user signs up via Google OAuth (**Auth**). A PostgreSQL database trigger automatically runs to create a record in `public.profiles`. The user then attempts to upload an avatar image to a private **Storage** bucket, which relies on a Storage **RLS** rule validating that `auth.uid() = bucket.owner_id`.
- **Root Cause & Chain:** The DB trigger failed silently due to an unindexed foreign key constraint during user creation. Consequently, the user exists in `auth.users` but not in `public.profiles`. When uploading to Storage, the Storage RLS policy attempts to reference profile attributes, fails, and returns an unhelpful 403 Access Denied error to the client.

2. Realtime Drops Updates Under High Load (Database + Connection Pooler + Realtime + RLS)

- **Scenario:** A live chat application uses **Realtime** subscriptions to stream incoming messages, protected by **RLS** policies ensuring users only see messages from their assigned rooms. During a user surge, messages stop arriving for active clients, though records continue inserting into PostgreSQL.
- **Root Cause & Chain:** Unindexed queries in the messages table exhaust direct PostgreSQL connections (**Database**). As CPU utilization hits 100%, PostgreSQL's Write-Ahead Log

(WAL) replication lags. The **Realtime** engine attempts to process WAL updates, but because every stream update must re-evaluate complex, unindexed **RLS** policy subqueries for hundreds of connected WebSocket clients, the Realtime engine times out and silently drops stream events.

3. Document AI Pipeline Timeout (Storage + Webhooks + Edge Functions + Vector DB)

- **Scenario:** A developer uploads a 50MB PDF document into a **Storage** bucket. A bucket webhook automatically calls an **Edge Function** to extract text, generate vector embeddings, and insert them into the PostgreSQL **Vector DB** (pgvector). The client application receives a 504 Gateway Timeout.
- **Root Cause & Chain:** The heavy PDF text parsing inside the **Edge Function** consumes significant wall-clock time, causing the function to hit its maximum execution timeout. Meanwhile, the partial batch of 1536-dimensional embeddings inserted into an unindexed pgvector table causes a sudden spike in database memory, locking the table and causing subsequent API requests across the platform to queue and fail.

4. Auth Refresh Loop Drains Connection Pool (Auth + Edge Functions + Supavisor)

- **Scenario:** A mobile application deployed to thousands of devices experiences a temporary network outage. Upon reconnecting, expired JWTs trigger rapid parallel token refreshes via **Auth**, which run through a custom **Edge Function** middleware for custom validation checks before reaching PostgreSQL.
- **Root Cause & Chain:** The Edge Function was configured to connect directly to PostgreSQL on port 5432 rather than using the **Supavisor** connection pooler on port 6543. The sudden stampede of thousands of mobile clients refreshing tokens concurrently launches thousands of direct DB connections, hitting the max_connections limit (57M01), crashing both the Auth service and all standard web database queries across the entire app.